

The Uredo manual

A Python-like surface syntax for Rust.

No runtime, no reimplementation — readable Rust, and Cargo builds it.

Language specification v0.4

2026-09-15

Every example marked uredo compiles; a test in the compiler's suite fails if one stops.



Contents

*	i
1. Who this is for	2
2. Install, and your first program	3
3. Read what it generated	4
4. Layout, comments, attributes	5
5. Bindings	6
6. Functions, and how arguments are passed	7
7. Types and literals	9
8. Optionals	10
9. Errors	12
10. Structs, enums, matching	13
11. Methods and traits	14
12. Iterators and closures	16
13. Modules and visibility	17
14. Using Rust crates, and the escape hatch	18
15. The toolchain	20
16. Debugging	21
17. Mistakes you will make	22
18. What Uredo does not do	25
19. Where to look things up	26

Uredo is a Python-like surface syntax for Rust. You write indentation and fewer sigils; the compiler emits readable, deterministic Rust and hands it to Cargo and rustc. There is no runtime, no garbage collector, and no Uredo-specific reimplementation of anything — the crate it generates is an ordinary Cargo package that a Rust programmer can read, review and ship.

This manual teaches the language. It is not the specification: `UREDOLANGUAGE_SPEC_v0.4.md` is a record of decisions and their evidence, written to be argued with, and it says so itself. When this manual states a rule, it cites the section that owns it, so you can go and read why.

Every example below marked `uredo compiles`, and a test in the compiler's suite (`compiler/tests/manual.rs`) fails if one stops compiling. Blocks marked `text` are fragments or error messages and are not compiled.

1. Who this is for

You know Rust, or enough of it to be dangerous, and you find its punctuation heavier than the ideas it encodes. Uredo keeps the ideas — ownership, borrowing, lifetimes, traits, **unsafe**, the error model — and spends fewer characters on them. Measured over twenty paired programs, it uses **13% fewer tokens** than the idiomatic Rust that does the same thing, and about half the annotations in signatures (§37).

It is a bad fit if you want Rust's semantics hidden. Nothing here removes the borrow checker; the diagnostics translate its complaints into Uredo's terms, which is a different service.

You do not need to know Rust's *syntax* especially well, because most of what Uredo changes is syntax. You do need to accept that ownership exists.

2. Install, and your first program

You need Rust (`rustc` and `cargo`) and `rustfmt`, since generated code is formatted before it is compared or compiled.

```
cd compiler && cargo build --release
export PATH="$PWD/target/release:$PATH"
```

Then:

```
uredo new hello
cd hello
uredo run
```

`uredo new` writes `Cargo.toml`, a `.gitignore`, and `src/main.ure`:

```
##! A new Uredo package. `uredo run` builds it and runs it.

fn main():
    name = "world"
    print("hello, {name}")
```

Three things are already visible:

- a block opens with `:` and is indented four spaces
- `##!` is a module doc comment
- `name = "world"` binds a new name — there is no **let** unless you want shadowing (§8.1)

`uredo run` lowered every `.ure` under `src/` into `target/uredo/`, wrote a provenance map beside it, and delegated to `cargo run`. Your package's `target/` holds the artefacts, as it would for any crate.

3. Read what it generated

Do this early and often. It is the fastest way to learn the language, and it is the project's whole claim — that the Rust it emits is code you would be willing to own.

```
uredo rust src/main.ure
```

```

//! A new Uredo package. `uredo run` builds it and runs it.

fn main() {
    let name = "world";
    ::std::println!("hello, {name}");
}

```

When you want to know *why* a line came out the way it did:

```
uredo explain src/main.ure:5
```

```

Expression:          print("hello, {name}")          src/main.ure:5:10
Uredo elaboration:   ::std::println!("hello, {name}")
    rule: $11.3 (`print` is an intrinsic: the literal is a Rust
               format string; other arguments are Uredo expressions)
Inserted by Uredo:   borrow: none · clone: none · allocation: none ·
                   dispatch: none · sync: none

```

(Trimmed: the real command also prints the Rust adjustments, the callee effects and the generated line.)

`explain` names the rule, and the rule is in the specification under that number. Everything Uredo inserts on your behalf — a borrow, an `Ok(...)`, a **self**. — is reported there, and there is a short list of things it will *never* insert: a clone, an allocation, a dispatch, a lock (\$26.2).

4. Layout, comments, attributes

The layout rules are short:

- indentation is four spaces
- tabs are an error
- a header line ends with `:`, and its body is the lines indented under it — or one expression on the same line

```
fn sign(n: i32) -> &'static str:
    if n > 0:
        "positive"
    else if n < 0:
        "negative"
    else:
        "zero"

fn shorter(n: i32) -> &'static str:
    if n > 0: "positive" else: "not positive"

fn main():
    print("{} {}", sign(-1), shorter(3))
```

Rust	Uredo
<code>// comment</code>	<code># comment</code>
<code>/// doc comment</code>	<code>## doc comment</code>
<code>//! module doc</code>	<code>##! module doc</code>
<code>#[derive(Debug)]</code>	<code>@derive(Debug)</code>
<code>#![no_std]</code>	<code>@!no_std</code>

Attributes are forwarded to Rust unchanged, so anything Rust accepts works, including a `derive` macro from a dependency (§23).

```
@derive(Debug, Clone, PartialEq)
struct Point:
    x: i32
    y: i32

fn main():
    print("{:?}", Point { x: 1, y: 2 })
```

Operators are Rust's, with Rust's precedence — `&&` and `||`, `not` and `and` or (§7.1). Paths use `::`; a `.` is field access, a method call, or `.await` (§6.7).

5. Bindings

Three spellings, and the difference between them is the whole of it:

- `name = expr` binds a new name
- `var name = expr` binds a mutable one
- `let name = expr` always creates a *new* binding, which is how you shadow

```
fn main():
    total = 1 + 2           # let total = 1 + 2;
    var count = 0           # let mut count = 0;
    count += 1
    let total = total * 10   # shadows the first `total`
    print("{total} {count}")
```

`=` means one of two things, and which one depends on what is already in scope (§8.1, D6):

- a name that already exists — an **assignment**, which requires **var**
- a name that exists nowhere in scope — a **binding**

The diagnostic tells you which you got when the two disagree.

A type annotation goes after the name, as in Rust:

```
fn main():
    items: Vec<u8> = Vec::from([1, 2, 3])
    print("{items:?}")
```

6. Functions, and how arguments are passed

This is the chapter to read twice. Everything else in Uredo is spelling; this is a rule you have to hold in your head.

A **parameter is a shared borrow unless its declaration says otherwise**. The compiler decides from the declaration alone — never from a type it would have to infer or look up in a dependency — and the call site gets the `&` inserted for it.

```
struct Job:
  name: String

fn describe(job: Job) -> String:      # &Job
  job.name.clone()

fn rename(job: inout Job, to: str):    # &mut Job, &str
  job.name = to.to_string()

fn consume(job: take Job) -> String:   # Job, moved
  job.name

fn main():
  var job = Job { name: String::from("build") }
  print("{} ", describe(job))          # describe(&job)
  rename(job, "test")                  # rename(&mut job, "test")
  # consume(job) – job is gone after this line
  print("{} ", consume(job))
```

The complete table is §10.2. The part worth memorising:

You write	Rust gets	Why
<code>x: Job</code>	<code>&Job</code>	the default: a shared borrow
<code>x: inout Job</code>	<code>&mut Job</code>	you intend to mutate it
<code>x: take Job</code>	<code>Job</code>	ownership moves to the callee
<code>x: u32, x: bool, x: char</code>	by value	known-Copy: copying is free and moves nothing
<code>x: str, x: [u8]</code>	<code>&str, &[u8]</code>	slices are already borrowed shapes
<code>x: &Job</code>	<code>&Job</code>	an explicit reference is emitted as written
<code>x: T, x: impl Trait</code>	by value	the type form is the visible sign of the transfer (D52)

take is the only annotation that marks a non-Copy move. That is deliberate and it is the rule the whole design turns on: reading a signature tells you whether a value survives the call.

Two other things move without the word — a type parameter ($x: T$) and a pattern parameter — and the *shape* is the sign in both cases.

`str` and `[T]` mean `&str` and `&[T]` in a parameter and in a return type, including under the optional suffix:

```
fn initial(name: str) -> char:
    name.chars().next().unwrap_or('?')

# Option<&str> in, Option<&str> out
fn domain(email: str?) -> str?:
    email?.split('@').nth(1)

fn main():
    print("{} {:?}", initial("ada"), domain(Some("ada@example.org")))
```

A return type is owned unless you write `str` or `[T]`, which borrow with Rust’s elided lifetime. If there is nothing to elide from — no borrowed input — you write the lifetime, and it will be `&'static str`:

```
fn label(flag: bool) -> &'static str:
    if flag: "on" else: "off"

fn main():
    print!("{}", label(true))
```

That is the one apostrophe real Uredo code tends to meet (D48, §9.7).

Calling into Rust

Arguments to a Rust function are Rust’s to spell. Uredo inserts a borrow only when it can resolve the callee’s signature, which means a function or method *you* declared in Uredo. For anything from `std` or a dependency, write what Rust wants:

```
fn main():
    text = String::from("hello world")
    # a std method, so no `&` is inserted
    parts: Vec<&str> = text.split(' ').collect()
    print!("{}", parts.len())
    var sorted = parts.clone()
    # `&b.len()` is Rust's, and you write it
    sorted.sort_by((a, b) => a.len().cmp(&b.len()))
    print!("{sorted:?}")
```

This is D2, and §5.2 explains why it cannot be otherwise: resolving a foreign signature would need a type engine, and Uredo deliberately has none. It is also the single largest source of `&` in real Uredo code, and the thing new users trip over most.

7. Types and literals

Scalars, tuples, arrays and slices are Rust's. Two things differ.

`[...]` is an array, not a **Vec**. A `Vec` is asked for explicitly:

```
fn main():
    fixed = [1, 2, 3]                # [i32; 3]
    grown: Vec<i32> = Vec::from([1, 2, 3]) # or Vec::from(...) / vec![...]
    print("{} {}", fixed.len(), grown.len())
```

Interpolated strings are for the four intrinsics only. `print`, `eprint`, `format` and `panic` take a literal that is a Rust format string; they capture identifiers and accept Rust's positional and named arguments (§11.3, D5).

```
fn main():
    who = "world"
    n = 3
    print("hello, {who}")
    print("{who} has {} items, {n} of them ready", n * 2)
    message = format!("{n:>4} | {who}")
    print("{message}")
```

Anywhere else, a string literal is just a string literal.

8. Optionals

`T?` is `Option<T>`. Values are `Some(x)` and `None` — Rust’s spelling; there is no `none`.

```
@derive(Debug)
struct Account:
    owner: String
    email: String?

## Two borrowed parameters, so elision cannot pick which one the
## result borrows from: the lifetime is written, exactly as it
## would be in Rust (§9.7).
fn find<'a>(accounts: &'a [Account], owner: str) -> &'a Account?:
    accounts.iter().find(a => a.owner == owner)

fn main():
    accounts = Vec::from([
        Account {
            owner: String::from("ada"),
            email: Some(String::from("ada@example.org")),
        },
        Account { owner: String::from("bob"), email: None },
    ])
    # an explicit `&'a [T]` is P6, so the caller writes the `&`
    if let Some(a) = find(&accounts, "ada"):
        print("{:?}", a.email)
    let Some(first) = accounts.first() else:
        print("no accounts")
    return
    print!("{}", first.owner)
```

A leading `&` binds tighter than the suffix, in every type position: `&T?` is `Option<&T>`, and a reference to the optional itself is `&(T?)` (§9.3, D54).

A parameter’s passing mode follows its *payload*, not its optionality:

```
# Option<u32>, by value: u32 is known-Copy
fn tier(score: u32?) -> &'static str:
    match score:
        Some(s) if s >= 90: "gold"
        Some(_): "member"
        None: "guest"

# &Option<String>: String is not known-Copy
fn describe(email: String?) -> &'static str:
    match email:
        Some(_): "has email"
        None: "no email"

fn main():
    print("{} {}", tier(Some(95)), describe(None))
```

9. Errors

A function that can fail is marked **throws**, and its return type is the success type. `throw` raises.

```
@derive(Debug)
struct TooBig

fn half(n: u32) -> u32 throws TooBig:
    if n > 100:
        throw TooBig
    n / 2

fn both(a: u32, b: u32) -> u32 throws TooBig:
    half(a)? + half(b)?

fn main():
    print("{:?} {:?}", both(8, 10), both(8, 200))
```

What that signature means, in three parts (§15, D8):

- `-> T throws E` is `Result<T, E>`
- `return e` and the tail expression are wrapped in `Ok(...)` for you
- `?` is Rust's operator, unchanged

A bare **throws** uses the crate's declared default error type. It is not allowed on a **pub** item, because a published signature must say what it returns (§15.1).

The one place `?` is not simply Rust's `is a tail or returned e?`, which lowers to a **match** rather than `Ok(e?)` so that a large error payload is not copied through a temporary (§15.3, D37). You will not notice, which is the point.

10. Structs, enums, matching

```
@derive(Debug, Clone)
enum Command:
    Move { dx: i32, dy: i32 }
    Say(String)
    Quit

fn describe(command: Command) -> String:
    match command:
        Move { dx, dy } if *dx == 0 && *dy == 0: String::from("stay")
        Move { dx, dy }: format("move by ({dx}, {dy})")
        Say(text): format("say {text:?}")
        Quit: String::from("quit")

fn main():
    commands = [
        Command::Move { dx: 1, dy: 0 },
        Command::Say(String::from("hi")),
        Command::Quit,
    ]
    for c in commands:
        print!("{}", describe(c))
```

An arm is `pattern: followed by one expression on the same line, or by an indented block`. Patterns are Rust's, guards included. Inside `describe` the `command` parameter is a `&Command` — the default borrow — which is why the fields arrive as references and `*dx` derefs one.

Variants are written without their type name in a **match** arm and with it in an expression, exactly as in Rust.

11. Methods and traits

Receivers are explicit. How you spell the receiver is its passing mode:

- **self** is **&self**
- **self: inout** is **&mut self**
- **self: take** is **self**, consumed
- no receiver at all makes it an associated function

That is §12.3 and D3. There is no implicit **self**: a field is `self.value`, and a method call on the receiver is `self.bump(1)`.

```
@derive(Debug)
struct Counter:
    value: i32

impl Counter:
    # no receiver: an associated function
    fn new(start: i32) -> Counter:
        Counter { value: start }

    # &self
    fn get(self) -> i32:
        value

    # &mut self
    fn bump(self: inout, by: i32):
        self.value += by

    # self, consumed
    fn into_value(self: take) -> i32:
        self.value

fn main():
    var c = Counter::new(1)
    c.bump(2)
    print("{} ", c.get())
    print("{} ", c.into_value())
```

Notice `value` in `get` with no **self**.: inside an instance method, a bare field name reads **self.field**, when it names a field of **Self** and nothing else of that name is in scope. It is read-only sugar — assigning needs `self.value` — and a glob **use** in the module switches it off, because Uredo cannot know what a glob imports (§12.2).

Traits are declared and implemented with the same spelling:

```
trait Describe:
  fn describe(self) -> String

  fn shout(self) -> String:           # a default method
    format("{}!", self.describe())

struct Dog

impl Describe for Dog:
  fn describe(self) -> String:
    String::from("a dog")

fn main():
  print!("{}", Dog.shout())
```

In an **impl** of a standard-library trait with a fixed signature — `From`, `PartialEq`, `Display`, the ops traits — the parameters take the modes the trait declares, whatever §10.2 would have chosen (D53). You write the signature the trait has; Uredo does not fight you about it.

12. Iterators and closures

A closure is `params => body`. One parameter needs no parentheses; several do.

```
fn main():
    numbers = [4, 8, 15, 16, 23, 42]
    evens: Vec<i32> = numbers.iter()
        .filter(n => *n % 2 == 0)
        .map(n => *n)
        .collect()
    total: i32 = numbers.iter().sum()
    biggest = numbers.iter().fold(0, (a, b) => if a > *b: a else: *b)
    print("{evens:?} {total} {biggest}")
```

Closure parameters are **Rust's**, not Uredo's: they take Rust patterns and optional type annotations, and **take/inout** do not apply to them (§17). `move x => ...` forces by-value capture.

`for x in xs` iterates by shared reference when `xs` is a place. To consume or to mutate, say so:

```
fn main():
    var words = Vec::from([String::from("b"), String::from("a")])
    for w in words:                # &String
        print("{w}")
    for w in inout words:          # &mut String
        w.push('!')
    for w in take words:           # String, consuming the Vec
        print("{w}")
```

That is D12, and it is why a loop that looks like it moves does not.

A long closure can open an indented block as the last argument of a call (D51):

```
fn main():
    names = Vec::from([String::from("ada"), String::from("bob")])
    lengths: Vec<usize> = names.iter().map(n =>
        trimmed = n.trim()
        trimmed.len()
    ).collect()
    print("{lengths:?}")
```

13. Modules and visibility

Visibility is Rust's: **pub**, **pub(crate)**, and private by default. A file under `src/` is a module, and Uredo writes the **mod** declarations for you (§20.3).

```
pub fn exported() -> u8:
    helper() + 1

fn helper() -> u8:                # private
    1

mod inner:                       # an inline module
    pub fn deeper() -> u8:
        2

fn main():
    print("{} {}", exported(), inner::deeper())
```

An automatically declared module is **private**. If you have both a `src/lib.ure` and binaries under `src/bin/`, a module the binaries need must be declared `pub mod name` in the library by hand — the binaries are separate crates and reach it by the crate's name.

use is Rust's, and `use rust::...` is not a thing; there is no separate namespace for Rust.

14. Using Rust crates, and the escape hatch

Dependencies go in `Cargo.toml` and are used as they are:

```
use std::collections::BTreeMap

fn tally(words: [&str]) -> BTreeMap<String, usize>:
    var counts = BTreeMap::new()
    for w in words:
        *counts.entry(w.to_string()).or_insert(0) += 1
    counts

fn main():
    print!("{:?}", tally(&["a", "b", "a"]))
```

When a construct has no Uredo spelling — an **extern "C"** block, an **unsafe impl**, a lang item, anything Uredo does not model — write Rust:

```
rust {
    pub unsafe fn from_raw(len: usize) -> usize {
        len * 2
    }
}

fn main():
    n = unsafe:
        from_raw(21)
    print!("{n}")
```

A **rust { }** block is passed through byte for byte, and its positions are preserved so that `file!`, `line!` and `include_str!` inside it see what they would in Rust (§25). It is not a failure mode; it is the documented boundary. §22 lists what belongs on the far side of it.

Uredo lowers each file **once**, for every target and feature set, so `@cfg` is forwarded and `rustc` selects (D50). The consequence to know: two declarations of one item under different `@cfg` must agree in their passing modes and receiver form, because the lowering reads declarations and cannot read your build configuration.

Declaring a macro

`macro_rules!` is Rust's and works here, which matters whenever a set of items is repetitive enough that writing them out is worse than writing the rule:

```
pub trait Field

macro_rules! scalar_fields {
    ($($t:ty),* $(,)? ) => { $(impl Field for $t {})* };
}

scalar_fields!(u8, i64, f64, bool, String)

fn accept<T: Field>(_value: &T):
    ()

fn main():
    accept(&1u8)
    accept(&String::from("x"))
```

The body is Rust — the same rule as every other macro (§22.5) — so what it generates is Rust items, not Uredo. That is right for the case above, where the output is a run of `impl` blocks. It also means a macro cannot factor out repeated **Uredo** syntax: there is no way to write one that emits `fn f(x: take T) -> U:` with an indented body, because by the time the macro is expanded there is no Uredo left to expand into.

So reach for it when the repetition is item-shaped, and reach for a trait with a bound when it is behaviour-shaped. Most repetition in Uredo turns out to be the second kind.

15. The toolchain

Command	What it does
<code>uredo new <path> [--lib]</code>	create a package
<code>uredo init [dir] [--lib]</code>	the same, in a directory that already exists
<code>uredo check [dir]</code>	lower, then <code>cargo check</code>
<code>uredo check --api [dir]</code>	compare the public API with <code>uredo-api.json</code>
<code>uredo build [dir] [--release]</code>	lower, then <code>cargo build</code>
<code>uredo run [dir] [-- args...]</code>	lower, then <code>cargo run</code>
<code>uredo test [dir]</code>	lower, then <code>cargo test</code> , doc examples included
<code>uredo doc [dir] [--open]</code>	lower, then <code>cargo doc</code>
<code>uredo fmt [--check] <paths></code>	canonical formatting
<code>uredo lint [--deny] [--allow L]</code>	idiom findings
<code>uredo lint --measure</code>	the counts §38's reversal trigger needs, not findings
<code>uredo fix [--dry-run] <paths></code>	apply the fixes that change no generated Rust
<code>uredo rust <file> [--map]</code>	print the generated Rust
<code>uredo explain <file>:<line></code>	what Uredo elaborated there, and why
<code>uredo package [dir] [--out p]</code>	export a self-contained Cargo package
<code>uredo publish [dir] [--dry-run]</code>	export, check the API and the licence, then <code>cargo publish</code>
<code>uredo lsp</code>	a language server on stdio
<code>uredo report [dir]</code>	a bug bundle for a lowering defect

Three of these are worth knowing about before you need them.

uredo check --api records your crate's public API in `uredo-api.json`. Every **pub** item has a signature derived from its declaration alone, so it cannot change when a dependency does (D14). The first run writes the file; later runs diff against it, and `uredo publish` refuses an unreviewed change, because a published API is a promise.

uredo fix applies only the findings whose rewrite provably leaves the generated Rust byte-identical — today that is `redundant_take` and nothing else. It checks that per edit and puts back anything that moves the output. Findings that change what a function *accepts* stay suggestions.

uredo report builds a bundle for a lowering defect: the source, the generated Rust, the provenance map and the diagnostic, with the failure classified by where the error landed. If the compiler emits Rust that does not compile, that is a bug in Uredo and this is how to report it (§5.4).

16. Debugging

Rust has no `#line` directive, so debug info names the generated `target/uredo/**/*.*rs`, and a debugger shows those lines. Uredo writes a provenance map beside them, and the helpers in `compiler/debug/` read it:

```
uredo build .
rust-gdb target/debug/hello
(gdb) source /path/to/compiler/debug/uredo_gdb.py
(gdb) ubreak src/main.ure:5
(gdb) run
(gdb) uwhere
(gdb) ulist
```

Three commands, the same three under LLDB:

- `ubreak FILE.ure:LINE` sets the breakpoint
- `uwhere` prints the backtrace in Uredo locations, with the source beside each frame
- `ulist` shows the Uredo source around the current frame

Stepping is still Rust's, and the editor's source view still shows the generated code; `compiler/debug/README.md` says exactly what is and is not translated.

Values need no translation: a Uredo binding is a Rust binding of the same name, except where §6.1 makes it a raw identifier — `gen` is `r#gen` to the debugger, because it is a Rust keyword.

17. Mistakes you will make

These are not invented. They are the errors made while writing 1,786 lines of Uredo tooling, in the order of how often they came up (`tools/README.md` keeps the full list), and sixteen more from writing `examples/restdemo` (its `README` keeps those).

Where a mistake below shows an error: with a section number in it, the compiler names the rule itself. That is deliberate and it is where the work goes: half of these used to arrive as a rustc type error about generated code, which is true, anchored to the right line, and no use at all if you do not already know the rule you broke.

Thinking `throws` says something about the value. Four of the thirteen were this one, and one of them was made at seven sites in a single function. `throws` names what a function returns when it **fails**, and says nothing about what it returns when it succeeds.

```
fn parse(text: str) -> u32 throws std::num::ParseIntError:
    text.trim().parse()?           # the tail is wrapped in `Ok` for you (§15.3)
```

So writing `Ok(...)` yourself returns `Ok(Ok(...))`. Uredo says so now, rather than letting rustc report a type error against your signature line:

```
error: a `throws` body wraps its tail in `Ok` for you (§15.3), so this returns
      ↪ `Ok(Ok(...))`
      Ok(1)
      ^
      = drop the wrapper and write the value; `throw e` or `e?` carries the error
      ↪ path
```

The mirror image is an optional return, which is **not** wrapped for you — only a `throws` body is, and it wraps in `Ok`:

```
fn first(items: take Vec<u32>) -> u32?:
    items.into_iter().next()      # already an Option; `Some(...)` here would be
    ↪ wrong
```

Get that one backwards — a tail that is a plain value under a `?` return — and the message names the asymmetry rather than leaving you with a type error:

```
error[E0308]: this function returns `i32?`, which is `Option<i32>`, and Uredo
  ↳ does not wrap a
      tail in `Some` for you
2 |     x
  |     ^ this is a `i32`
= help: write `Some(...)` around it, or return `None`
= note: `throws` does wrap a tail, and wraps it in `Ok` (§15.3); `T?` has no
  ↳ such rule (§14)
```

A third in the same family: a `throws` tail that is **already** a `Result` would be wrapped twice, and wants `?`. Uredo says so. (A fourth is still uncaught: `throws` with `no ->` is a **unit** return, so a body that yields a value has it discarded.)

Forgetting that arguments to Rust functions are Rust's. By a wide margin the most common.

```
error[E0308]: mismatched types
  .captures_iter(row.body)
      ^^^^^^^^^ expected `&str`, found `String`
= note: arguments of `captures_iter` are written as Rust wants them
```

Write `&row.body`. Uredo inserts the borrow for *its own* callees only (D2).

Writing and for &&. Uredo's layout is Python's; its operators are Rust's.

Putting the passing mode before the name.

```
error: the passing mode goes on the type: write `name: take T` (§6.1)
```

Returning a `String` parameter. A `String` parameter is a borrow, so there is nothing to return:

```
error[E0308]: `s` is a parameter declared `s: String`, which is a shared borrow
  ↳ (§10.1) —
      there is nothing here to give away
= help: declare it `s: take String` to take ownership, or write `s.clone()`
```

Write `take String` when you mean to consume it.

Giving a `for` binding away. `for w in words` iterates by shared reference (D12), so `w` is a `&String` and a `take` parameter will not accept it. Write `for w in take words` to consume the collection — the message says so, and names D12, because a loop that looks like it moves does not.

Expecting `String?` to mean `Option<&String>`. `str` and `[T]` mean `&str` and `&[T]` under `?`; `String` and `Vec<T>` do not. So `-> String?` against `v.first()` is a mismatch, and the fix is `.cloned()` or a return type the elision rule accepts (§9.7).

A slice of `str`. `str` means `&str` where a parameter is one, not inside a slice of them: write `[&str]`, not `[str]`.

Deferred initialisation. There isn't any: `var x: usize` with no value is not a binding. Compute the value as an expression instead, which usually reads better anyway.

`Option::unwrap_or_else` takes no argument where `Result`'s takes one. That is Rust's distinction, and `rustc`'s message arrives on your Uredo line.

A two-parameter closure needs parentheses: `(a, b) => ...`, not `a, b => ...`. Without them the first parameter reads as an argument to the call, which `rustc` can only report as a name it cannot find — so Uredo says what it actually is:

```
error[E0425]: `a` reads as an argument here, but it looks like the first
    ↳ parameter of a closure
3 |     t: i32 = v.iter().fold(0, a, b => a + *b)
  |                                   ^ parsed as an argument
  = help: a closure with more than one parameter needs parentheses: write `(a,
    ↳ ...) => ...` (§17)
```

Naming a binding `crate`, `self`, `Self` or `super`. Every other Rust keyword is escaped to `r#name` for you; these four have no raw form, and Uredo says so rather than emitting something that will not compile.

18. What Uredo does not do

Uredo (§39):

- does not replace Cargo, crates.io, rustc or LLVM
- adds no garbage collector and no runtime ownership fallback
- invents no async runtime
- hides neither ownership, lifetimes, **unsafe**, traits nor the error model
- adds no class hierarchy, and none of Python's dynamic semantics
- wraps no crate in a Uredo-specific API

The escape hatch is Rust itself.

19. Where to look things up

The specification is organised so that you can go straight to the rule:

Question	Section
Lexical structure, identifiers, keywords	\$6
Expressions and operator precedence	\$7
Statements and bindings	\$8
Types in signatures, references, lifetimes	\$9
Passing modes, the call-site rule	\$10
Program structure, strings and interpolation	\$11
Structs, methods, receivers, field sugar	\$12
Enums and pattern matching	\$13
Optionals	\$14
Error handling	\$15
Loops, closures	\$16, \$17
Generics, traits	\$18, \$19
Modules and visibility	\$20
Async	\$21
Rust interop and <code>rust { }</code>	\$22
Attributes	\$23
<code>unsafe</code>	\$24
The lowering contract	\$25
Memory and performance transparency, <code>explain</code>	\$26
Diagnostics	\$27
Toolchain and versioning	\$28
Formatting and lints	\$29
What is decided, held and open	\$38
What Uredo will not become	\$39

\$0 is the record: every decision, why it was taken, what was measured, and what would reverse it. It is worth reading once you disagree with something — the argument is probably already in there, with the evidence that settled it.